

Voice Assistant Architecture Specification

A living software architecture specification for a local-first conversational AI assistant.

1. Vision

Build a private, local-first, real-time conversational assistant whose primary goal is natural human conversation while remaining modular, maintainable, and future-proof.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

2. Guiding Principles

Local-first operation, modular design, interface-driven architecture, streaming everywhere, deterministic infrastructure, explainable configuration, graceful degradation.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

3. High-Level Architecture

Describe the assistant as cooperating services: Audio I/O, STT, Conversation Engine, Speech Rewrite, TTS, Memory, Context Manager, Personality Manager, Tool Framework, Configuration, Telemetry.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

4. Voice Pipeline

Audio capture → Voice activity detection → Streaming STT → Incremental LLM generation → Speech rewrite → Streaming TTS → Audio playback.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

5. Low Latency

Overlap all stages, stream tokens, synthesize sentence-by-sentence, avoid unnecessary serialization.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

6. Speech-to-Text

Use MLX Whisper or equivalent. Support endpoint detection, partial transcripts, punctuation recovery, and confidence estimates.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

7. Conversation Engine

Support interchangeable LLM providers through one interface. No application code should depend directly on a model implementation.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

8. Prompt Architecture

Separate system prompt, personality prompt, task prompts, tool prompts, and speech rewrite prompts.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

9. Speech Rewrite

Convert written language into spoken language while preserving intent. Remove markdown, lists, and unnecessary verbosity.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

10. Text-to-Speech

Treat TTS as an interchangeable module. Support multiple voices, expressive speaking styles, and configurable prosody.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

11. Voice Profiles

Represent timbre, pacing, warmth, pitch tendencies, pauses, and expressiveness independently of personality.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

12. Personality

Represent humor, curiosity, directness, empathy, verbosity, confidence, and teaching style as configurable traits.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

13. Memory

Layered memory: working memory, episodic memory, semantic memory, and user preferences. Retrieval should be selective.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

14. Context Orchestration

Determine which memories, prompts, tools, and conversation history are injected each turn.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

15. Interruptions

Support barge-in, cancellation, recovery, and resumable responses.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

16. Configuration

Store every configurable subsystem as versioned JSON/YAML profiles.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

17. Swappable Models

All models implement a shared interface. Replacing a model should require configuration changes only.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

18. Tool Framework

Expose optional tools behind standardized interfaces so the conversational core remains independent.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

19. Observability

Collect latency metrics, token counts, prompt versions, failures, memory retrievals, and quality metrics.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

20. Evaluation

Maintain benchmark conversations, latency tests, voice evaluations, regression suites, and A/B prompt testing.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

21. Failure Handling

Every subsystem fails independently with graceful degradation and detailed diagnostics.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

22. Security & Privacy

Keep processing local by default. Explicitly distinguish offline and online capabilities.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

23. Roadmap

Phase 1: conversational core. Phase 2: memory. Phase 3: tools. Phase 4: specialist agents. Phase 5: ecosystem.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

24. Future Directions

Emotion recognition, multimodal input, adaptive personalities, collaborative agents, continuous benchmarking.

Design Consideration 1. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 2. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 3. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 4. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 5. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

affecting unrelated modules.

Design Consideration 6. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 7. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.

Design Consideration 8. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules. This section should define responsibilities, interfaces, inputs, outputs, extension points, performance expectations, testing strategy, implementation notes, operational considerations, and future evolution. Every subsystem should own one responsibility, expose a stable interface, avoid hidden dependencies, and remain replaceable without affecting unrelated modules.